Tushe × Noema — Response to the Synthetic Enterprise Environment requirements

Responding to: *NOEMA × TUSHE | Synthetic Enterprise Environment — Technical Requirements & Acceptance Specification · v1.0*
From: Tushe engineering
Date: 7 October 2026
Covers: all 73 numbered requirements (63 P0 · 10 P1), all 20 acceptance tests (§22), the 20-scenario minimum pack (§23), and the twelve direct questions in §24.
Describes: the sandbox as it runs today.

How to read this

This document has two halves, and you only need the first one.

Part One is the answer in plain language: what we can do, what we can't, where we think your spec is wrong, and what we need from you. It runs about three pages and assumes no knowledge of our system.

Part Two is for your engineers. It walks every one of your 73 numbered requirements, all 20 acceptance tests and all 20 scenarios, one row at a time, and says exactly where each one stands. It is deliberately dry. If your team wants to argue with us, that is the half to argue with.

Part One — the answer

1. The short version

You asked whether we can give your agents a realistic banking environment to be tested against, including one that misbehaves on purpose. Yes — most of it exists today, we built more of it after reading your spec, and there are two things in your spec we think you should drop.

Here is the whole thing in four points.

1. The hard part is already built. The single most important thing you asked for is that our system should be able to say "we sent the money, and the recipient hasn't got it" — and keep saying it, rather than tidying the contradiction away. That is now how our payouts work. A payment carries three separate verdicts: what *we* did, what the *bank rail* did, and whether the *recipient was actually credited*. They are allowed to disagree, and when they disagree, the disagreement is the answer. Your agents can see all three.

2. We call you, and we can tell you every time we tried. Everything that happens is recorded in a stream your systems can read, re-read and replay after an outage — that is the authoritative record — and a notification also arrives at your server the moment it happens, signed so you can prove it came from us.

The part that matters for your testing: we keep a full delivery log. *Did you send me that notification, when, how many times, and what did my server say back?* — your team can now answer that for any event, including ones that failed. And you can replay any delivery on demand, which is also how you test a duplicate arriving.

What is still outstanding is making notifications fail **on purpose** — dropping one, delaying one, putting two out of order to order. That is about two weeks on top of what now exists, and it is the only remaining item your acceptance tests depend on.

3. Two of your requirements would make the environment worse, not better. One asks us to build a master clock you can fast-forward. Nobody in payments does this — Stripe, Flutterwave, Treasury Prime and Lead Bank all control

time one transaction at a time instead — and it directly contradicts another of your own requirements, that two of your developers should be able to run tests at the same time without corrupting each other. We offer the approach the industry actually uses.

The other asks us to give your test harness a special back-door view of the "true" answer, so you can mark your agent against it. We can build that. We think you'll regret it, and this is the one place we'd push back hardest: the published research on testing AI agents treats that back door as a known way for benchmarks to get cheated, with named cases where agents found the answer key instead of doing the work. **What you actually need is a test environment you can reset to an identical starting state and replay — which is exactly what we built.** Your harness then works out the right answer itself, where your agent can't reach it.

4. We're adding something your spec is missing, and it matters here. When a Nigerian transfer goes quiet, a bank doesn't guess and doesn't resend. It asks the rail again, in a specific way, with specific timing, for up to 24 hours. That procedure has a name — Transaction Status Query — and it is the single most important habit an agent working on Nigerian payments has to learn. It isn't in your spec. It is now in our product, and your agents can practise against it.

What this costs you in time

Nothing is blocked. The sandbox is running now, at the version this document describes, and your team can be working against it **this week**. The pieces that still need building run from late October to mid-December, in the order your acceptance tests need them. The detail is in §11.

The four things we need you to decide

1. **Cases and disputes** — your spec mentions them; we don't have them, and we think that's your product rather than ours. In scope for us or not?
2. **The back-door answer key** — read §5.2 and tell us your call. If you still want it, we'll build it and note it as a deliberate exception.
3. **The "impossible state" scenario** — we'd like to replace it with something achievable and, we think, more realistic. §5.5.
4. **One word in your acceptance criteria** — "the same *material* fault sequence". Nobody has defined *material*, and if we leave it undefined it becomes an argument on acceptance day rather than a test. §5.4.

One thing to know before your engineers start

Our test environment is honest about being a test environment, not a mock. Everything your agents do runs through the same code that moves real money, which is the point — but it also means we won't fake a state our real system can't produce. Where that bites, we say so plainly in §9, and we would rather over-disclose now than surprise your team in week three.

Part Two — the detail, for your engineers

2. How to read the status column

STATUS	MEANING
Supported	Available today. Demonstrable in a joint session on request.
Supported differently	The capability is met, by a different mechanism than the one specified. We say which, and why.
Requires extension	The substrate exists and the work is defined. Sized in §7.
Not supported	Not available, and not proposed for the POC window. The reason is stated.

STATUS	MEANING
Challenged	We believe the requirement as written is wrong, under-specified, or in conflict with another of your requirements. A counter-proposal is included.

A note on what "Supported" means here. It means built, tested and merged — not designed. Everything marked Supported has unit and end-to-end tests that drive it through the real application against a real Postgres, and the money-moving paths additionally have multi-step scenario tests that assert on ledger balances rather than response codes. Where something is built but not yet in the shared sandbox, we say so.

Release state. Everything described in §4 is **released and running on the shared sandbox**. Nothing in the "Supported" column is waiting on a release, and nothing in this document describes something that exists only on a branch.

---

3. §24 — your twelve questions, answered directly

YOUR QUESTION	ANSWER
Which logical domains (§3) exist natively today?	Customers & identity, accounts & ledger, transactions/payments, settlement, beneficiaries, events, policy/authorization and audit — all native. **Cases/disputes do not**, and we want to ask you about that rather than guess (§8).
Which failure/chaos behaviours (§7) are available natively?	**Application and state faults**: declined, failed-after-accept, reversed-after-completion, never-resolved, delayed, and commit-then-no-answer. **Not** transport faults, and **not** event-delivery faults.
Which behaviours would need custom work or a control hook?	Event delivery faults (drop/duplicate/delay/reorder), stale reads, ambiguous name enquiry, and a diagnostics plane that labels injected vs genuine. Transport faults we propose you **buy**, not have us build (§6).
Can you provide an isolated, privileged ground-truth read path (§15)?	**We can, and we recommend you do not take it.** See §5.2 — this is our strongest disagreement with your document, and it is sourced.
Can you provide deterministic scenario IDs and seeded, replayable execution (§7–8)?	**Yes.** Scenario key + version + seed, a SHA-256 checksum of the definition in every run artefact, and object IDs derived as UUIDv5 of `(partner, key, version, seed, ref)` — so the same scenario and seed produce the same identifiers, not merely the same shapes.
Can you provide a controllable virtual clock (§9)?	**No, and we think you should not want one.** We provide per-object scheduled time and manual advance, which is what Stripe, Flutterwave, Treasury Prime and Lead Bank all do. See §5.1.
Can you expose independent ledger, transaction, switch, settlement and beneficiary state (§3)?	**Yes.** A payout carries three independent lifecycles that are permitted to disagree — our record of the instruction, the rail's settlement, and whether the beneficiary was credited — over the double-entry ledger. Switch state is the one difference: it is carried by the transition log's source attribution rather than a fifth column (SE-01).
Can you preserve cross-system inconsistencies without normalising them (§14)?	**Yes**, and it is enforced rather than merely allowed: nothing in the system reconciles the three lifecycles, and the one place that could have — the watchdog — is explicitly forbidden from acting on an unconfirmed outcome.
Can you provide event loss / delay / duplication / reordering controls (§10)?	**Duplication and replay, yes** — any delivery can be replayed on demand and the attempt log makes it traceable. **Deliberate loss, delay and reordering, not yet** — that is injecting faults into a delivery path that now exists, which is E3.
Can you provide reset / snapshot / restore / replay (§8, §15)?	**Reset and replay, yes.** Snapshot/restore, no — and we think seed + reset + replay makes it unnecessary. Say if you disagree.
Can you provide isolated environments for parallel testing (§16)?	**Yes, per partner tenant and per environment**, which is a real boundary in the database, not a convention. Two parallel runs inside *one* tenant and environment are not isolated from each other — the practical answer is one tenant per concurrent run, which we can provision.

YOUR QUESTION	ANSWER
Expected delivery dates for the P0 capabilities?	§7.
What known limitations should we document before integration?	§9. We would rather over-disclose these now.

4. What exists today, and why

Six pieces of work sit behind most of the "Supported" rows. In plain terms they are: a payment that can hold three contradictory verdicts at once; a readable record of everything that happened; the ability to ask the bank “what actually became of this?”; fake account numbers that make a payment fail on purpose; a one-command reset to a known starting state; and signed notifications pushed to your server with a full record of every attempt.

Each is described below at the level your engineers will want. Each carries an internal architecture decision record — the design, the alternatives weighed and the open items — and we are happy to share any of them under the NDA if the reasoning matters to you.

4.1 Three independent payout lifecycles — *released, v1.32.0*

Your §5 insists that transaction status and settlement status must not collapse into one field. We agree, and we model it on the payments standard rather than on a convention of our own: ISO 20022's `pain.002` ladder, where `ACSP` (accepted, settlement in process) guarantees strictly less than `ACSC` (accepted, settlement completed). A single status field cannot express "we sent it, and the recipient does not have it", and that distinction is the one your document is built around.

A payout carries:

LIFECYCLE	STATES	WHAT IT ANSWERS
`status`	`pending · processing · completed · failed · reversed · unknown`	What *we* did with the instruction
`settlementStatus`	`not_required · pending · settled · failed · reversed · unknown`	What the *rail* did
`arrivalStatus`	`not_applicable · pending · arrived · not_arrived · unknown`	Whether the *beneficiary* was credited

They are three columns, written independently, and **nothing synchronises them**. `completed` + `not_arrived` is a legal, reachable, queryable state — it is debited-but-not-credited, which is the condition your whole document is about. Every change to any of them writes a row to a transition log carrying the previous state, the new state, the source (`api · provider_webhook · watchdog · operator · backfill`), the provider's own reference, whether the transition was inferred rather than observed, a reason, the trace ID, and both `occurredAt` and `observedAt` .

This is a production capability every partner with a support desk needs, and it is what satisfies SE-01, SE-04, STM-01, REC-03 and REC-05.

4.2 The event feed — *released, v1.32.0*

Every integration event is written to a per-partner feed carrying an event ID, module, resource, action, entity ID, the resource's public representation as of that moment, a **monotonic per-partner sequence number**, the trace ID, the causation ID of the internal domain event it was translated from, and `occurredAt` . Pagination is cursor-based on `sequence` , not on timestamp — two events in the same millisecond have no defined order, and a cursor that can repeat a row is how a consumer double-processes.

The feed has two doors with different reach. An **API key** sees `GET /v1/events`, and only events in modules the key holds a read scope for — the feed is not a bypass for per-module scoping. A **partner administrator**, authenticating with a dashboard session rather than an API key, sees `GET /partner/events` across every module. An API key cannot reach the second door at all. That asymmetry is what we offer in place of GT-02, and it is the subject of §5.2.

Why pull first, and what it leaves. This ordering is deliberate, and it follows where the market has moved rather than where it started. Stripe's current direction is *thin events*: the webhook carries an ID and a type, and the receiver fetches the authoritative object from the API — which makes the queryable feed the source of truth and the push a notification about it. The standard integration pattern is webhooks as the primary path with a queryable feed as the reconciliation backup, and the backup is the part that is hard to retrofit, because it requires durable ordering and a cursor that cannot repeat a row.

It is also the part this market mostly lacks. Paystack's webhook delivery log is a dashboard view, not an API; Flutterwave's replay is `POST /v3/transactions/{id}/resend-hook`, one transaction at a time. Neither offers "give me every event since sequence N", which is what a consumer actually needs after an outage — and which we have.

The push half — signed outbound delivery with retries and an attempt history — sits on top of it and is described in §4.7.

4.3 A payout we could not confirm is not a payout that failed — *released, v1.34.1*

Your UNK-04 — "the environment must not silently convert an unknown transport outcome into a confirmed success or failure" — is the requirement we think is hardest to satisfy honestly, and the place where a payments platform most often gets it wrong.

The trap is that a failure to *get an answer* looks, in code, exactly like an answer of *no*. Treat them the same and a timeout on a transfer that was in fact accepted returns the customer's money while the beneficiary also gets paid — the double payment this whole class of system exists to prevent.

So our initiation path separates two outcomes that are not the same outcome:

- **Definitive.** The provider answered, and the answer was no. Nothing was created. Reversing the customer's debit is correct.
- **Ambiguous.** A timeout, a connection failure, a `5xx`, a `408` / `425` / `429` from an intermediary that has no idea what the provider behind it did with the request, or a success response whose transfer identifier we cannot read. The request may well have become a transfer. **Nothing reverses.** A watchdog is armed instead, and the payout stays in an unknown state until something authoritative resolves it.

The same distinction runs through the status query and the webhook path: `unreachable` is never mapped to `failed`. The same caution applies to provider vocabulary — a status such as `abandoned`, which neither PSP documents as terminal, is **not** treated as a failure, because failure routes straight into a compensating credit and an unverified status has no business on that branch.

For your purposes this is the behaviour your agents can rely on: an unknown outcome stays unknown, visibly, until it is resolved — and it is resolved by asking, never by assuming.

4.4 Magic values and manual advance — *released, v1.34.1*

In **test mode only**, a destination account number of the form `9999 SS MMMM` selects a simulated outcome: `SS` is the scenario, `MMMM` is a delay in minutes before it is applied.

CODE	SCENARIO	WHAT IT PRODUCES
`999900MMMM`	success	Accepted and completed, after the delay
`999901MMMM`	decline	The provider answers *no*. The only path on which the debit reverses
`999902MMMM`	unreachable	No answer at all. The debit stands, nothing reverses — **this is your UNK-01**

CODE	SCENARIO	WHAT IT PRODUCES
`999903MMMM`	fail_after_accept	Accepted, then reported failed
`999904MMMM`	reverse_after_complete	Accepted, completed, **then reversed by the rail** — the three lifecycles disagree and the disagreement is the fact
`999905MMMM`	stay_pending	Accepted and never resolved. The watchdog path

Test mode is what makes this safe, and it is not a caveat bolted on: in test mode we never reach a real bank, so a number that means something to us cannot mean something to someone else. In live these are ordinary account numbers with no special meaning, and the interception code returns `null` before it reads a digit.

Alongside it, `POST /v1/accounts/{accountId}/payouts/{payoutId}/simulate/advance` applies the scenario immediately rather than waiting out the delay. It returns 404 in live — and that is a property of the route, not a claim in its description: it checks both that the payout is a test payout and that the *caller* is in test mode, so a live key presenting a test payout ID cannot drive the simulated lifecycle.

4.5 Sandbox scenarios — seeded, versioned, resettable — *released, v1.34.1*

```
GET  /v1/sandbox/scenarios                 list built-ins and your own
POST /v1/sandbox/scenarios                 define one
POST /v1/sandbox/scenarios/{key}/apply     materialise it
POST /v1/sandbox/scenarios/{key}/reset     undo it
```

A scenario definition names customers, accounts, balances, KYC tiers and currencies. Four built-ins ship with it (`solo-funded-customer`, `tier-ladder`, `cooperative-5`, `empty-account`); partners may define their own, versioned.

Three properties matter for your purposes:

- **Determinism is in the identifiers, not just the shapes.** Every object's ID is `UUIDv5(partner : key : version : seed : ref)`. Apply the same scenario with the same seed and you get the same customer IDs and the same account IDs, so a test can hard-code them.
- **The definition is checksummed.** SHA-256 over a stable serialisation, returned with every listing. This is your API-02 applied to the scenario pack rather than the API — a scenario quietly edited between runs otherwise silently changes what a grade means, and the benchmark literature's recommendation on this is blunt: freeze environments at release time.
- **Reset is complete.** It deletes the scenario's objects *and* the whole ledger transactions that funded them. Apply → reset → apply returns the environment to a funded baseline, which is what makes a scenario replayable rather than single-use.

This design follows Plaid, whose `sandbox-custom-users` is a public repository of seeded JSON scenario definitions and whose Sandbox Studio lets a team pick a pre-built test user from a dashboard. The catalogue is for everyone; the definition format is for teams who want their own.

4.6 Transaction Status Query — the thing your §13 is missing

When a Nigerian transfer's outcome is unknown, a bank does not guess and does not retry the transfer. It **requeries**. NIP returns `09` ("request processing in progress — try again"); the bank waits roughly 40 seconds, sends a Transaction Status Query to NIBSS, retries up to three times at five-second intervals, and may continue querying for up to 24 hours. The standing instruction is *never reverse a debit without confirming the final status through TSQ*, and CBN requires the refund inside 24 hours or the bank is fined.

Your UNK-03 asks for "enough independent evidence to investigate an unknown outcome". On Nigerian rails, that evidence has a name and a shape, and an environment without it trains an agent in a generic investigation loop rather than the Nigerian one — which defeats the stated purpose of your §13.

So we built the equivalent: `POST /v1/accounts/{accountId}/payouts/{payoutId}/status-query`.

```
outcome: terminal | pending | unreachable | cooling_down
```

Four design points, each of which an agent can be taught against:

- It **asks the provider**, keyed on *our* reference — because a payout whose initiation never confirmed has no provider-side identifier, and that is precisely the payout most worth asking about.
- It **never reverses**. `unreachable` means no answer, which means no state change. `pending` means the provider says the transfer is still being worked — that is an answer meaning "ask again later", not a failure.
- It **writes through the same code path a webhook does**, so there is no second implementation of the lifecycle to drift from the first.
- It is **rate-limited per payout** with a cooldown, and returns `cooling_down` with a `retryAfterSeconds` rather than silently coalescing. A recovery operation that itself returns an unresolved answer is your scenario 13, and it is reachable here without any special configuration.

We should be straightforward about one limit: **we are not a NIBSS participant — your partner's PSP is.** What we query is the PSP's transfer-verification endpoint, and the semantics differ by provider. Paystack's `GET /transfer/verify/{reference}` is keyed on our reference and can therefore answer about a transfer we never got confirmation for. Flutterwave's `GET /v3/transfers/{id}` is keyed on *their* identifier, which we do not have in exactly that case — so on Flutterwave the ambiguous payout is the one the query cannot resolve. That is a real asymmetry in the rails, not a defect in our model, and an agent trained here should learn it.

4.7 Signed webhooks, and a record of every attempt — *released, v1.34.1*

This is what turns the feed from something you poll into something that reaches you.

```
POST   /partner/webhooks/endpoints              register a URL, choose your events
POST   /partner/webhooks/endpoints/{id}/rotate-secret
GET    /partner/webhooks/messages               the delivery log
POST   /partner/webhooks/messages/{id}/replay   send one again
POST   /partner/webhooks/endpoints/{id}/recover requeue everything that failed since a time
```

Six things about it are relevant to what you are building.

- **The notification is thin, and points back at the record.** The body carries an id, a type, and a URL — not the resource. Your receiver fetches the authoritative event from the feed. This is Stripe's *thin events* model, and it matters here because it means a notification can never disagree with the record: the two are not two copies of the truth, they are a pointer and the thing it points at.
- **Signing is Standard Webhooks, verbatim.** HMAC-SHA256 over the raw body plus a timestamp, with a five-minute tolerance, so a captured delivery cannot be replayed at you later. Your team can verify with an off-the-shelf library rather than code written from our prose. Rotation signs with both the old and new secret for 24 hours, so a key change never drops a delivery.
- **★ Every attempt is recorded and readable by you.** Outcome, your HTTP status, a truncated copy of what your server returned, and how long it took — retained past the retry window, because its value is during the post-mortem rather than during the incident. This is EVT-06, and it is the question the feed alone could not answer.
- **Delivery is at-least-once with an eight-step ladder** over roughly 28 hours, then the message is marked failed and stays replayable. An endpoint that fails continuously for five days is disabled and you are told. Note the five days keys on *elapsed time*, not a failure count: a quiet endpoint must not be able to stay broken indefinitely simply by receiving less traffic.

- **Replay keeps the same delivery id**, so a receiver that de-duplicates recognises something it already processed. That is also how you produce your scenario 5 — a duplicate notification — today, without us having to build fault injection first.
- **We will not call an address we have not vetted.** HTTPS only, and the hostname is re-resolved and range-checked before *every* delivery, not once at registration. We then connect to that vetted address rather than resolving a second time, while still validating your certificate against your hostname. Worth knowing because it means a webhook URL pointing at a private or internal address is refused and logged as `blocked`, rather than quietly attempted.

What this does not yet do is fail on purpose. Dropping a notification, delaying one, or delivering two out of order are faults injected into a delivery path that now exists — roughly two weeks of work, and the only remaining dependency for three of your acceptance tests.

5. Where we disagree with your document

Five items. Each is a judgement we would rather have out in the open now than discover at acceptance.

Each one opens with the point in a sentence; the evidence follows for whoever wants it.

5.1 The fast-forward clock (TIM-01, TIM-02)

> **In a sentence:** you asked for a master clock you can wind forward. Nobody in payments builds one, it breaks another of your own requirements, and our database would contradict us if we claimed to have one. We offer the thing the industry actually uses instead.

Our position: decline as written, and we believe the requirement is wrong on its own terms.

First, it contradicts your DATA-05. "The control plane can freeze and advance simulated time" implies one clock for the environment. DATA-05 requires that "parallel developer/CI runs must not corrupt one another's state". One run advancing a global clock corrupts every concurrent run. Both are P0; they cannot both hold as written.

Second, nobody in this market ships one. We looked, because "we can't" is a weaker answer than "nobody does, and here is what they do instead":

PROVIDER	TIME CONTROL
Stripe	Test clocks, **scoped to the objects in a simulation** — not to the platform
Flutterwave	Settlement delay encoded per transaction (`DU_{minutes}`)
Treasury Prime	`scheduled_settlement` in minutes, plus a `manual` flag
Lead Bank	A per-object `advance` endpoint

Third, we would be lying if we said yes. Fifty-one columns in our schema default to `now()` in Postgres. An application-level clock is overruled by the database on every one of them, so a "global virtual clock" from us would be a half-truth — correct in the application, wrong in the rows an agent then reads.

What we offer instead, which is the same mechanism all four providers above use: **time in the domain's own units, per object.** The `MMMM` field of a magic account number schedules an outcome N minutes out; `POST .../simulate/advance` fires it on demand. This satisfies TIM-04 ("delayed settlement, delayed credit and event delivery must be schedulable and replayable") and AT-15's pass condition ("scheduled transitions fire according to simulated time") without a platform clock existing at all. It also preserves DATA-05, because advancing one payout does not touch another run's.

5.2 The back-door answer key (GT-02)

> **In a sentence:** you want a privileged view of the "true" outcome so you can mark your agent. We can build it. The research on testing AI agents says putting the answer key inside the thing being tested is how benchmarks get cheated — and what you actually need instead is something we have already built.

Our position: we can build it, and we recommend against it. The reason is not cost.

Your GT-02 asks for "a privileged read that establishes the actual underlying outcome, independent of whatever response the integration client received", and GT-03 requires that ordinary credentials cannot reach it. The instinct — that the grader must see what the agent cannot — is exactly right. The *placement* is what we would change: **on the platform the agent is driving, rather than in the harness that grades it.**

What the canonical benchmark actually does. τ-bench grades by comparing final database state to a ground-truth target — and it does not ask the environment for that target. It **replays the ground-truth write actions on a fresh database copy, hashes the result, and compares it to the hash of the actual database after the agent ran.** The ground truth is *computed by the harness*, from known-correct actions against a known seed. The environment supplies state and tools. It does not supply a verdict.

And the literature makes it a rule. *Establishing Best Practices for Building Rigorous Agentic Benchmarks* (arXiv:2507.02825) lists it as check **T.5 — "fully isolate agents from the ground truth results"** — and names what happens when it is violated:

- **KernelBench** "failed to remove ground truth answers from GPU memory, allowing agents to obtain the correct result through out-of-bounds memory access."
- **SWE-Lancer**: "agents can locate the tests and replace them with a trivial assertion… achieving a perfect score without solving any intended tasks."

Both are cases where the oracle lived inside the environment the agent could act on. An agent on our platform holds credentials, makes requests, and — in any realistic version of your product — will eventually hold credentials with more reach than you first issued. A privileged read that exists on the same platform is one credential-scope mistake away from being the answer key.

What makes the canonical method available to you is precisely what §4.5 describes: a **seeded, resettable, deterministically replayable** environment. Replay your known-correct action sequence against a fresh apply of scenario S at seed K; hash the resulting state; compare. The IDs are derived from the seed, so the comparison is exact rather than structural. That is τ-bench's method, running in your harness, where the isolation actually holds.

And for the parts of GT-01/GT-04 that are genuinely about observation rather than grading — the actual sequence of state changes, the injected conditions, the full cross-module history — we already have the privileged surface: `GET /partner/events` behind a partner-administrator session, which no API key can reach, plus the transition log and the append-only audit trail. An explicit test proving the API-key refusal ships with E1, so the isolation is demonstrable rather than asserted.

If after reading this you still want GT-02 as specified, say so and we will build it — it is your benchmark. We would want the decision recorded as a deliberate exception rather than a default.

5.3 Marking only the final state (SCN-04)

> **In a sentence:** if you grade an agent purely on where things ended up, an agent that correctly says "I don't know, escalate" scores the same as one that did nothing at all — and on the investigations you care about, those are the two outcomes you most need to tell apart.

Our position: the mechanism is fine; the grading model it implies is the known-gameable one.

"Each scenario retains the authoritative final state needed to grade a run" is τ-bench's design, and the best-practices review found that **a trivial agent passes 38% of τ-bench tasks with no domain knowledge**, because the ground truth rests on trivially-reachable states.

For an agent whose job is *investigating a messy outcome*, final state is an especially weak signal. An agent that correctly concludes "unknown — escalate, do not re-send" and an agent that does nothing at all land on the same end state. Your document's own most important requirement — UNK-04, never normalise — describes a situation where the *right* answer is to change nothing.

What distinguishes them is **process**: did it query the switch separately? did it check settlement independently of transaction status? did it re-send a payment whose outcome was unknown? And **reliability across repeated trials** — τ-bench's `pass^k`, because an agent that gets it right once in four is not an agent that gets it right.

Neither appears anywhere in your document. Our transition log, event feed and audit trail are what make process grading possible, and we would rather raise this than quietly supply the inputs for a grading model we think is weak.

5.4 One undefined word (CHA-02)

> **In a sentence:** your acceptance test says a re-run must produce "the same *material* fault sequence". Nobody has said what *material* means, and on acceptance day that single word decides whether a three-millisecond timing difference is a pass or a fail. Let us agree it now, in writing, while it is cheap.

Our position: deliverable, but the word *material* is load-bearing and currently undefined.

Deterministic Simulation Testing — FoundationDB, TigerBeetle, WarpStream, Antithesis — gets determinism by running the system single-threaded in simulation with every source of nondeterminism (clock, message latency, loss, ordering, thread scheduling, RNG) behind one seeded PRNG, so a whole run is a pure function of one 64-bit seed. TigerBeetle is a financial transactions database built this way. **Your instinct is drawn from a real discipline and it is the right instinct.**

A hosted multi-tenant REST service cannot do it. Concurrent tenants share a process; Postgres has its own ordering and its own clock; connection pools and HTTP arrival order are not under our control.

What we can deliver honestly:

DETERMINISTIC	NOT DETERMINISTIC
Seed data — identical objects and identical IDs from `(scenario, version, seed)`	Wall-clock durations and inter-request timing
The fault *schedule* — same scenario and seed, same injected faults, same logical order	Interleaving with other tenants' work
The authoritative outcome of each object	Row-level ordering where no explicit sort applies
The scenario definition itself — checksummed	

Our proposal for AT-01's pass condition, as a joint definition: *the set of injected faults, their targets, their logical order relative to each other, and the final authoritative state of every scenario object reproduce exactly. Wall-clock timings and inter-run latencies do not.*

5.5 The "impossible state" scenario (19)

> **In a sentence:** you asked us to create a state our system is specifically built to make impossible. We can only do that by switching off the safety rail — which stops the environment resembling the real one — or by faking it on screen, which teaches your agent to distrust the wrong thing. Your own spec already asks for something better.

Our position: challenge, with a counter-proposal from your own document.

Your STM-02 requires us to document which states must never coexist in authoritative state. Your §23 then asks us to create one. For a mock this is trivial. For a real platform standing in for production, the two available routes are both bad:

- **Disable the `CHECK` constraint.** The environment stops representing the system it stands in for, and an agent learns against a platform with different invariants than the real one.
- **Fake it at the read layer.** Then it is a *display* inconsistency, not an impossible *state* — and it teaches an agent to distrust the wrong thing: our read API rather than the reality underneath it.

Chaos engineering as practised perturbs **availability and timing**, never **correctness invariants**, and we think that distinction is right rather than merely conventional.

The counter-proposal is already in your document. REC-05 asks for conflicting source records that remain independently retrievable and that the environment must not resolve. That is both achievable and *more* realistic: in production, an agent never meets a record that is internally impossible — it meets two systems that each hold an internally-consistent record and disagree with each other. The three payout lifecycles produce exactly that, by construction: `status: completed` with `arrivalStatus: not_arrived` is two systems disagreeing, each correct from where it sits, with nothing reconciling them.

We propose replacing scenario 19 with a cross-system contradiction scenario, and we would rather say so plainly than quietly ship something that looks like scenario 19 and is not.

---

6. One thing to buy rather than have us build

> **In a sentence:** making the network itself misbehave — dropped connections, timeouts, gateway errors — is a solved product category you can buy off the shelf. Us hand-building it inside our payment service would be slower, worse, and would put deliberately broken code in the path that moves money.

Transport faults — connection timeout, disconnect, reset, 502/503/504, delayed response (your §7's transport class, UNK-02, parts of CHA-03, AT-04).

A service-virtualization proxy — **WireMock Cloud, Hoverfly or Microcks** — sitting in front of the sandbox does all of this, configurably, deterministically, today. Building a connection-reset simulator inside a NestJS application reinvents a mature product category and, worse, puts deliberate misbehaviour inside the money path. We do not want code in our payment service whose purpose is to break it.

The one exception, which stays ours: UNK-01, commit-then-timeout. That is not a transport fault. The commit has to really happen *in our system* and the response has to then be lost, which a proxy in front of us cannot arrange. It is built, at our API edge, as magic scenario `02` (§4.4).

---

7. Requirement-by-requirement

Statuses as defined in §2. P1 rows are marked; your §25 requires P0s only.

§3 Systems and boundaries

ID	REQUIREMENT	STATUS	NOTE
SE-01	Independent system state	**Supported**	Ledger, transaction (`status`), settlement (`settlementStatus`), beneficiary credit (`arrivalStatus`) are four independently queryable dimensions. *Switch* is not a fifth column: it is represented by the transition log's `source` and the provider's own reference. If you need it as a separate queryable state, say so — it is an additive change.
SE-02	Cross-system divergence	**Supported**	`reverse_after_complete` produces it on demand; `completed + not_arrived` is the standing case.
SE-03	Correlation	**Supported**	One trace ID propagates request → payout → transition → event → audit. The rail is joined by our deterministic reference `payout-{id}` and the provider's transfer reference, both stored.
SE-04	No forced convergence	**Supported**	Nothing synchronises the three lifecycles. The one component that could — the watchdog — is explicitly forbidden from acting on an unconfirmed outcome.

§5 State machines

ID	REQUIREMENT	STATUS	NOTE
STM-01	Transition contract	**Requires extension**	The states, triggers and terminal states exist and are enforced; the *document* you need — lifecycle diagrams with triggers and invariants — is a deliverable we owe, not code. Included in E1.
STM-02	State invariants	**Requires extension**	Partly enforced by database `CHECK` constraints today. Same deliverable as STM-01. See also §5.5.
STM-03	Transition history	**Requires extension**	**The data is complete and the read surface is missing.** Every transition is written with from-state, to-state, source, source reference, inferred flag, reason, trace ID, `occurredAt` and `observedAt`. There is no endpoint that returns them yet. This is the single cheapest high-value item in the list. E1.

§6 Unknown outcomes — "*the single most important behaviour in the environment*"

ID	REQUIREMENT	STATUS	NOTE
UNK-01	Post-commit unknown	**Supported**	Magic scenario `02`. The debit commits; the client gets no usable answer; nothing reverses. Deliberately ours rather than the proxy's (§6).
UNK-02	Pre-commit uncertainty	**Requires extension**	*Ambiguous* pre-commit transport failure belongs to the proxy layer (§6). What we have natively is the *definitive* decline (scenario `01`), which is the opposite case.
UNK-03	Evidence retrieval	**Supported**	The TSQ-equivalent status query (§4.6), the event feed, the ledger, and the transition log once E1 exposes it.
UNK-04	Never normalise	**Supported**	`unknown` is a first-class state on all three lifecycles; `unreachable` is never mapped to `failed`; the watchdog never reverses on an unconfirmed outcome; an unverified provider status falls through to pending rather than to failure. §4.3.

§7 Deterministic chaos / failure injection

ID	REQUIREMENT	STATUS	NOTE
CHA-01	Scenario ID	**Supported**	Scenario key + version + seed + definition checksum.
CHA-02	Deterministic seed	**Supported, with a limitation we want agreed**	Deterministic seed data, fault schedule and authoritative outcome. Not deterministic execution — no hosted multi-tenant service can offer that. Proposed joint definition of "material" in §5.4.

ID	REQUIREMENT	STATUS	NOTE
CHA-03	Scripted faults	**Partially supported**	"Commit then return 504" — **yes**. "Deliver Y twice" — **yes**, via replay. "Drop event X" and "return stale state for N seconds" — **no**: deliberate loss is E3, stale reads are the proxy (§6).
CHA-04 *(P1)*	Scoped faults	**Requires extension**	Today: per-object (the destination account number) and per-scenario. Not per-endpoint or per-system.
CHA-05	Hidden injection	**Requires extension — and a design difference worth discussing**	Our magic values are *chosen by the integration client*, which is how Flutterwave and Modern Treasury do it and is right for ordinary partner testing. Your requirement is for faults configured in the control plane and **not** disclosed to the client. That is a different mechanism, and the scenario engine is the right place for it. E3.
CHA-06	Fault trace	**Partially supported**	A simulated transition is distinguishable in the data (`SIMULATED_*` reasons, `TRF_SIM_` references, the `source` column), but there is no diagnostics surface that reports what was injected, when, against what target, and what the underlying outcome was. E5.

§8 Scenario engine

ID	REQUIREMENT	STATUS	NOTE
SCN-01	Scenario definition	**Partially supported**	Initial state, resources, seed, version and checksum — **built**. Injected faults, event schedule, timing and invariants as part of the *definition* — **E4**. (They are configurable per object today, just not declared in the scenario.)
SCN-02	Reset & replay	**Supported**	No manual database intervention. Reset removes the scenario's objects and the ledger transactions that funded them.
SCN-03	Composite failures	**Requires extension**	One fault per object today; several objects in one scenario can each carry one, which is a partial answer. Ordered and simultaneous composition is E4.
SCN-04	Authoritative outcome	**Challenged**	§5.3. The retention mechanism is fine; we think the grading model it implies is the gameable one, and we would rather you grade process and `pass^k`.

§9 Time

ID	REQUIREMENT	STATUS	NOTE
TIM-01	Virtual clock	**Not supported — declined, with sourced reasoning**	§5.1. Fifty-one `now()` database defaults; nobody in the market ships one.
TIM-02	Freeze & advance	**Challenged**	§5.1. Contradicts your own DATA-05. We offer per-object advance instead.
TIM-03	Temporal metadata	**Supported**	`occurred`, `processed`, `posted`, `settled`, `arrived`, `observed` — all present. `delivered` **does not exist**, because outbound delivery does not exist (E2).
TIM-04	Scheduled transitions	**Supported differently**	Scheduled per object in minutes, fired by a durable workflow; manually advanceable. Satisfies the requirement without a platform clock.

§10 Events

ID	REQUIREMENT	STATUS	NOTE
EVT-01	Event schema	**Partially supported**	*(unchanged by the webhook release — `deliveredAt` now exists on a delivery, but causation is still not exposed on the event itself)*

ID	REQUIREMENT	STATUS	NOTE
EVT-02	Duplicate delivery	**Supported, in part**	*The same event delivered twice* — **yes**: replay re-sends a delivery keeping the same id, so a de-duplicating receiver sees exactly the case it must handle. *The same business effect under two different event ids* — not reproducible on demand; E3.
EVT-03	Ordering	**Partially supported**	Sequence metadata is retained and is the pagination cursor. Deliberate *reordering* needs delivery (E3).
EVT-04	Loss, delay, replay	**Partially supported**	**Replay is built** — one message or everything that failed since a given time. Deliberate *loss* and *delay* are fault injection on a path that now exists: E3.
EVT-05 *(P1)*	Authenticity	**Supported**	Standard Webhooks signing — HMAC-SHA256 over the raw body with a timestamp and a five-minute tolerance, so a captured delivery cannot be replayed at you later. Zero-downtime secret rotation signs with both keys for 24 hours. Verifiable with off-the-shelf libraries.
EVT-06	Delivery history	**Supported**	Every attempt is recorded and readable by you: outcome, your HTTP status, a truncated copy of your response body, and the duration — retained past the retry window. §4.7.

§11 Idempotency & concurrency

ID	REQUIREMENT	STATUS	NOTE
CON-01 *(P1)*	Concurrency control	**Not supported**	No ETag or version token on resources today.
CON-02	Concurrent modification	**Partially supported**	On money paths, concurrency is handled by single-statement conditional updates backstopped by database constraints, so a concurrent mutation is deterministically rejected rather than racing — two simultaneous payouts cannot both pass a daily limit. A *generic* read-then-stale-mutate scenario needs CON-01. E6.
(§11 idempotency)	Idempotency contract	**Supported**	Tenant-and-environment-scoped `Idempotency-Key` store with a request fingerprint and a snapshot of the first response. A retry replays it; a same-key-different-body reuse is a deterministic 422 with no second effect. Partner A and Partner B may use the same key independently.

§12 Authorization & policy

ID	REQUIREMENT	STATUS	NOTE
AUTH-01	Business authorization	**Supported**	Authenticated-but-unauthorized is a rich surface: per-module scopes, module-enablement gates, KYC-tier caps, daily debit limits, customer-type scoping. Each returns a stable, documented business code.
AUTH-02 *(P1)*	Approval states	**Partially supported**	Pending/approved/rejected exist in business onboarding and KYB review. There is no approval workflow on payouts specifically. E6.
AUTH-03	Policy mutation	**Supported**	Control policies are versioned with status and effective state. A dedicated endpoint answers "what policy applies, as of when", and a backtest endpoint answers "what would this policy have done to the history" — which also covers MUT-04.
AUTH-04	Tenant / resource isolation	**Supported**	Partner context is resolved and enforced on every request; test and live are a database-level dimension on balances, ledger entries, payouts and events, not a flag in application code. Cross-tenant failure is deterministic and is covered by end-to-end tests.

§13 Nigerian rail semantics

ID	REQUIREMENT	STATUS	NOTE
NG-01	Rail semantics	**Supported**	NUBAN, name enquiry, NIP references, reversal semantics — behind provider adapters, documented in the developer portal and the OpenAPI spec.
NG-02	Reference propagation	**Supported**	Our deterministic reference and the provider's transfer reference both persist on the payout, on every transition, and in events.
NG-03	Response taxonomy	**Supported, and extended**	Confirmed failure versus unresolved outcome is a *modelled* distinction, not a documentation convention: `definitive` on the initiation path, `unreachable` versus `failed` on the query path. §4.3 and §4.6.

§14 Evidence, reconciliation & timeline

ID	REQUIREMENT	STATUS	NOTE
REC-01	Transaction history	**Requires extension**	Recorded in full; needs the STM-03 read surface. E1.
REC-02	Timeline	**Requires extension**	The components exist across the transition log, ledger and event feed. A single correlated timeline endpoint is E1.
REC-03	Evidence by source	**Supported in substrate**	The `source` column (`api · provider_webhook · watchdog · operator · backfill`) plus `sourceRef` keeps evidence attributed and uncollapsed. Exposed by E1.
REC-04	Reconciliation surface	**Supported**	Machine-readable reconciliation reports in JSON and CSV, already shipped for statements.
REC-05	Preserve contradictions	**Supported**	By construction — see SE-04. This is also our counter-proposal for scenario 19 (§5.5).

§15 Ground truth

ID	REQUIREMENT	STATUS	NOTE
GT-01	Authoritative state	**Supported**	Commits, events and scenario configuration are all authoritative and durable; the scenario definition is checksummed.
GT-02	Privileged oracle	**Declined as specified — counter-proposal in §5.2**	We will build it if you decide you want it, recorded as an agreed exception.
GT-03	No ordinary-client access	**Supported**	An API key cannot reach partner-administrator routes. An explicit refusal test ships with E1 so the boundary is demonstrable on request.
GT-04	Run trace	**Supported differently**	The event feed (with sequence), the transition log and the append-only audit trail together are the run trace. Causation exposure is E1.
GT-05 *(P1)*	Export	**Requires extension**	E7 — a run-artefact export bundling scenario definition, checksum, seed, objects, transitions and events.

§16 Data

ID	REQUIREMENT	STATUS	NOTE
DATA-01	Synthetic only	**Supported**	Test mode is a first-class dimension in the database, enforced on balances, ledger entries, payouts and events — not a separate deployment that could drift from production.
DATA-02	Deterministic seed data	**Supported**	Seed → derived UUIDv5 identifiers → identical IDs across applies.

ID	REQUIREMENT	STATUS	NOTE
DATA-03 *(P1)*	Scale tiers	**Requires extension**	Four built-ins today, all small. Generating medium and large tiers is definition work on an existing engine. E4.
DATA-04	Adversarial data	**Requires extension**	Today's built-ins are clean. Duplicate customers, similar and ambiguous names, multiple matching accounts, repeated amounts, stale records and conflicting metadata are **scenario definitions**, not new engine work. E4 — and we agree with your framing that a trivial single-record lookup must never be sufficient.
DATA-05	Isolation	**Supported, with a boundary we want stated**	Isolation is per partner tenant **and** per environment, at the database level. Two parallel runs inside *one* tenant and environment share state. The practical answer is one tenant per concurrent run; we can provision these.
DATA-06	Reset	**Supported, scoped**	Scenario reset removes everything a scenario created, with no manual database work. There is no *whole-environment* reset to a baseline; we think seed + reset is the better primitive, but tell us if it is not.

§17 API & integration contract

ID	REQUIREMENT	STATUS	NOTE
API-01	OpenAPI	**Supported**	Generated from the code and regenerated with every change that touches a route, published alongside the developer portal.
API-02	Versioning	**Supported, and extended to scenarios**	`/v1` with a backward-compatibility policy, plus a version and checksum on every scenario definition — which answers the gap we found in your document: the API is versioned but the scenario pack was not.
API-03	Error contract	**Supported**	Documented HTTP statuses and stable business codes. Retryability is explicit rather than implied, via the definitive/ambiguous distinction.
API-04	Correlation	**Supported**	One trace ID across API, event and audit records.
API-05 *(P1)*	Pagination & filtering	**Supported**	Cursor pagination on a monotonic sequence, with documented filters.

§18 Audit & provenance

ID	REQUIREMENT	STATUS	NOTE
AUD-01	Immutable audit	**Supported**	Append-only audit records with before-state and after-state.
AUD-02	Transition provenance	**Partially supported**	The transition log carries source, source reference, previous and new state, reason, inferred flag, trace ID and both timestamps. **Actor and actor type live on the audit record, not the transition** — the join is the extension. E1.
AUD-03 *(P1)*	Policy provenance	**Supported**	Policy version and effective state are recorded and queryable.
AUD-04	Event provenance	**Partially supported**	Causation is recorded on every event; it is not exposed on either read surface. E1.
AUD-05	Audit query	**Partially supported**	Indexed by partner and time today. Retrieval by transaction, account, customer and correlation ID is E1.

§19 Observability

ID	REQUIREMENT	STATUS	NOTE
OBS-01	Diagnostic visibility	**Requires extension**	"*Was that our bug or the simulator?*" is answerable from the data today — simulated transitions carry distinguishing markers — but there is no diagnostics surface that says so plainly. We think this is one of your more important requirements and it deserves a real endpoint rather than an inference. E5.

ID	REQUIREMENT	STATUS	NOTE
OBS-02	Fault visibility	**Partially supported**	**Delivery history is built** (§4.7). The injected-fault half — a diagnostics surface that says plainly "this was us, deliberately" — is E5.
OBS-03 *(P1)*	Latency	**Requires extension**	Request latency is logged, not exposed. Configured-versus-actual timestamps need E5.

§20 Mutation safety

ID	REQUIREMENT	STATUS	NOTE
MUT-01	Independent permissioning	**Supported**	Scopes are **derived from the module registry** rather than hand-listed, and evaluated literally — `accounts:read` and `accounts:write` are separate grants, and a write scope on one module confers nothing on another. Money-moving scopes are flagged as such and can be gated on a second factor.
MUT-02	Simulation integrity	**Supported**	Every simulated outcome is applied through the same code path a real provider webhook takes. A compensating credit in a simulation is a real double-entry ledger posting with a real audit record — there is no shadow path that behaves differently.
MUT-03	Initial read-first access	**Supported**	We will issue your first credentials with read scopes only.
MUT-04 *(P1)*	Preview capability	**Supported for policy; not general**	"Would this be allowed, and what would it do" exists for control policies as a simulate/effective/backtest trio, and the simulation **shares the enforcing code** rather than mirroring it — a preview that reimplements the rule is a preview of a different system. There is no dry-run on payout creation.

8. §22 acceptance tests, and §23 scenario pack

Acceptance tests

ID	TEST	TODAY	NOTE
AT-01	Deterministic replay	**Partial**	Seed data and authoritative outcome reproduce. Needs the joint definition of *material* (§5.4).
AT-02	Ground-truth isolation	**Pass**	An API key cannot reach the privileged surface. Explicit test shipping in E1.
AT-03	Post-commit timeout	**Pass**	Magic scenario `02`.
AT-04	Stale state	**Fail**	Proxy layer (§6), plus a read-staleness profile. E6.
AT-05	Event loss	**Partial**	A delivery that fails is recorded as such and is visible in the log. *Dropping* one deliberately is E3.
AT-06	Duplicate event	**Pass**	Replay re-sends with the same delivery id, and both attempts appear in the log — "configured duplicate deliveries occur and are traceable".
AT-07	Out-of-order event	**Fail**	Sequence metadata is retained and deliveries are real; deliberate *reordering* is E3.
AT-08	Cross-system conflict	**Pass**	The three lifecycles, which never reconcile.
AT-09	Ambiguous beneficiary	**Fail**	Name enquiry returns a single name today. E6.

ID	TEST	TODAY	NOTE
AT-10	Idempotency conflict	**Pass**	Same key, different body → deterministic 422, no second effect.
AT-11	Retry after timeout	**Pass**	Scenario `02` then retry with the same key.
AT-12	Authorization boundary	**Pass**	Denied with a traceable business code.
AT-13	Policy mutation	**Pass**	Both versions and their effective times are queryable.
AT-14	Concurrent update	**Partial**	Deterministic on money paths; generic version-token detection is E6.
AT-15	Time control	**Pass, differently**	Per-object advance rather than a global freeze (§5.1).
AT-16	Reset & replay	**Pass**	No manual reconstruction.
AT-17	Ground-truth export	**Fail**	E7.
AT-18	Mutation isolation	**Pass**	Read-only credentials deny every monetary mutation.
AT-19	Audit completeness	**Partial**	All fields exist; actor and transition are in two tables. E1.
AT-20	Simulator diagnostics	**Fail**	E5.

Today: 12 pass, 6 partial, 2 fail. The two remaining failures are deliberate event reordering (E3) and run-artefact export (E7).

Scenario pack

#	SCENARIO	TODAY	HOW
1	Normal successful transfer	**Yes**	`999900 0000`
2	Failed before commit	**Yes**	`999901 0000` — the definitive decline, the only path that reverses
3	Commit then timeout	**Yes**	`999902 0000`
4	Debited / not credited	**Yes**	`completed` + `arrivalStatus: not_arrived` — the state the lifecycle split exists for
5	Duplicate webhook	**Yes**	Replay re-sends the same delivery id; both attempts are in the log
6	Missing webhook	**Yes**	`999905` (never resolves), then the status query resolves it — the TSQ scenario
7	Out-of-order webhook	**Partial**	`999904` delivers a reversal *after* a completion — real causal disorder, and now really delivered. Arbitrary reordering is E3
8	Stale read	**No**	E6 / proxy
9	Contradictory evidence	**Yes**	The three lifecycles. We propose this as the replacement for 19
10	Delayed settlement	**Yes**	The delay field — `999900 0060` settles in an hour
11	Beneficiary ambiguity	**No**	E6
12	Duplicate submission	**Yes**	Idempotency store

#	SCENARIO	TODAY	HOW
13	Recovery uncertainty	**Yes**	The status query itself returns `unreachable` or `cooling_down`
14	Unauthorized principal	**Yes**	Scopes, module gates, tier caps
15	Approval required	**Partial**	Exists in onboarding and KYB, not on payouts. E6
16	Policy changed	**Yes**	Versioned control policies with effective-time queries
17	Concurrent modification	**Partial**	Deterministic on money paths; generic case is E6
18	Malformed / stale event	**No**	E3
19	Impossible state	**Challenged**	§5.5 — we propose 9 in its place and will not fake it
20	Composite failure	**Partial**	Several objects each carrying a fault; ordered composition is E4

Today: 11 executable, 5 partial, 3 requiring extension, 1 challenged.

---

9. Known limitations — the things we would rather over-disclose

Your §25 requires these documented before integration. We would rather give you more than you asked for — everything below is something your team would otherwise find in week three.

1. **Notifications cannot yet be made to fail on purpose.** Signed delivery, the attempt log and replay are live (§4.7). What is missing is injecting a deliberate drop, delay or reordering — E3, and the only remaining dependency for AT-05 and AT-07.
2. **The payout transition log has no read endpoint.** The data is complete and has been since the lifecycle split; nothing returns it. E1.
3. **Causation IDs are recorded and not exposed.** A deliberate earlier decision about internal detail, which your EVT-01 and AUD-04 make the wrong call for this use. E1.
4. **Parallel-run isolation is per tenant and environment, not per run.** Two concurrent runs in one tenant share state. Use one tenant per concurrent run.
5. **Reset is scenario-scoped, not environment-wide.** There is no "restore the whole sandbox to baseline" operation.
6. **No snapshot/restore.** We believe seed + reset + deterministic replay subsumes it. Challenge us if your harness needs point-in-time restore.
7. **Magic values are client-selected, not control-plane-hidden.** Your CHA-05 wants faults the integration client cannot see. Ours are chosen by the caller. Different mechanism, same engine — E3.
8. **Status query semantics differ by provider.** On Paystack it is keyed on our reference and can resolve an unconfirmed payout. On Flutterwave it is keyed on their identifier, which we do not have in exactly that case. We have marked the rate-limit behaviour of both verify endpoints as *must-verify against a live sandbox* — neither provider documents one, and we will not claim a number we have not observed.
9. **We are not a NIBSS participant.** TSQ semantics are modelled; the actual query goes to the PSP. An agent trained here learns the right shape, mediated by a PSP rather than by NIBSS directly.
10. **No case/dispute domain.** See §10.
11. **The scenario engine seeds customers, accounts and balances — not transaction histories.** DATA-03's scale tiers and DATA-04's adversarial fixtures are both real work (E4), not configuration we have and have not turned on.
12. **Everything in §4 is released and running on the shared sandbox.** There is nothing in the "Supported" column that is merged-but-not-deployed. We state this explicitly because "built" is not the same claim as "running where you can

reach it".

10. What we need from you

1. **Cases and disputes.** Your §3 lists a case/dispute domain and your scenario pack does not exercise one. We do not have it, and case resolution looks like *your* product rather than infrastructure we should be modelling. **Is it in scope for us, and if so, what does the minimum look like?** We would rather ask than design it.
2. **GT-02.** Read §5.2 and tell us your decision. If you still want it, we build it and record it as an agreed exception.
3. **Scenario 19.** Accept the cross-system contradiction counter-proposal, or tell us the invariant violation is essential and we will discuss which of the two bad routes is less bad.
4. **AT-01's "material".** Agree the definition in §5.4 — or propose another — before acceptance rather than during it.
5. **Concurrency model.** How many parallel runs do you expect, and are you content with one tenant per run?
6. **Which P1s, if any, matter to you.** Your §25 requires P0s only, so we have sized them out. Three of them — EVT-05 authenticity, GT-05 export and CON-01 version tokens — are cheap once their P0 neighbours land, and we would rather include them than negotiate them later.

11. Sequencing and indicative dates

Delivery is expressed in **sessions**. A session is how we actually plan and ship: one self-contained piece of work with its own tests, code review, documentation and release check, start to finish — roughly three to five working days. We quote in sessions rather than dates because a session either lands complete or it does not land. Dates below assume a start on **13 October 2026** and are indicative, not contractual, until §10's questions are answered.

ID	WORK	SERVES	SESSIONS	INDICATIVE
E1	**Read surfaces over data we already hold** — transition-history endpoint, correlated timeline, causation exposure, audit query by correlation/entity/account, the explicit API-key-refusal test, and the §21 state-model and lifecycle documentation	STM-01/02/03, REC-01/02/03, AUD-02/04/05, EVT-01, GT-03/04, AT-02/19	1–2	**24 Oct**
~~**E2**~~	~~**Outbound event delivery**~~ — signed delivery with retries, attempt history, `deliveredAt`, and a partner-facing delivery log	EVT-05/06, TIM-03, part of EVT-02/04 and OBS-02, AT-06	—	✅ **Already built** — §4.7
E3	**Event-delivery faults, control-plane configured and hidden from the integration client** — drop, delay, reorder, malformed. *(Duplicate is already reachable via replay.)*	EVT-03/04, CHA-03/04/05, AT-05/07, scenarios 7/18	2	**7 Nov**
E4	**Scenario definition richness** — adversarial fixtures, scale tiers, faults and event schedules declared in the definition, ordered composition	SCN-01/03, DATA-03/04, scenario 20	2	**21 Nov**
E5	**Diagnostics plane** — injected versus genuine, fault trace, configured-versus-actual timings, latency	CHA-06, OBS-01/02/03, AT-20	1	**28 Nov**
E6	**The remainder** — ambiguous name enquiry, payout approval states, version tokens and stale-mutation detection, stale-read profile	CON-01/02, AUTH-02, AT-04/09/14, scenarios 8/11/15/17	2	**12 Dec**
E7	**Run-artefact export**	GT-05, AT-17	0.5	with E5
Buy	**Service-virtualization proxy** in front of the sandbox	UNK-02, transport faults, part of CHA-03 and AT-04	—	procurement, not engineering

The honest read on this sequence. E1 and E2 are the items we would build regardless of this engagement — E1 only exposes data we already collect, and E2 closed a standing production gap for every partner rather than only for you. E3 through E6 are shaped by your requirements specifically, and we would want them scoped against the acceptance tests you actually intend to run rather than against the full list.

We can start E1 immediately. Everything in §4 is available for a joint working session now; a shared sandbox tenant with read-only credentials and the four built-in scenarios can be provisioned this week.

---

12. Closing

Your specification is a good one — better than most we are sent, and specific enough to build against. We have answered it in the same spirit: directly, with the gaps named rather than smoothed over.

The capabilities your requirements lean on hardest are in our product today: a payment that can hold three contradictory verdicts at once, a clear line between “it failed” and “we don’t know”, the ability to ask the bank what actually happened, a sandbox you can reset and replay, and signed notifications with a full record of every attempt.

That is also why we have pushed back where we have. The two declines in §5.1 and §5.2 are not us trimming scope: a global clock would be a half-truth our own database would contradict, and a ground-truth oracle on the platform the agent is driving is the documented way these benchmarks get gamed. In both cases there is a better-sourced alternative, and in the second case **the alternative is a capability we have already built**.

What we would like next is simple: an hour with your team on the four decisions in §10, and read-only credentials in your engineers’ hands this week. Everything in §4 is ready to be shown working.

Sources for the claims in §4.2 and §5

- τ-bench — ground truth computed by the harness: known-correct writes replayed against a fresh database copy, hashed and compared. Also the source of the 38% trivial-pass finding and of `pass^k`.
- *Establishing Best Practices for Building Rigorous Agentic Benchmarks*, arXiv:2507.02825 — check **T.5**, "fully isolate agents from the ground truth results"; KernelBench and SWE-Lancer as named violations.
- **Stripe** test clocks (object-scoped); **Flutterwave** per-transaction settlement delay and failure catalogue; **Treasury Prime** `scheduled_settlement` and `manual`; **Lead Bank** per-object `advance`; **Plaid** `sandbox-custom-users` and Sandbox Studio; **Modern Treasury** magic counterparty account numbers.
- **Stripe** thin events — the webhook carries an ID and type; the receiver fetches the authoritative object from the API. The documented integration pattern is webhooks primary, a queryable feed for reconciliation.
- **Paystack** webhook delivery log (dashboard only, no API); **Flutterwave** `POST /v3/transactions/{id}/resend-hook` (per transaction). Neither exposes "every event since sequence N".
- **ISO 20022** `pain.002` status ladder — `ACTC → ACCP → ACSP → ACSC`.
- Deterministic Simulation Testing — FoundationDB, TigerBeetle, WarpStream, Antithesis.
- **NIBSS / CBN** — NIP response code `09`, Transaction Status Query timing (~40s, three retries at 5s, up to 24 hours), and the 24-hour refund obligation.